

Relatório Técnico: Engenharia de Dados, Modelagem Dimensional e Otimizações de Performance – Projeto COVID-19 (ES)

Lorena Moreira de Nadai

22 de junho de 2026

Resumo

Este relatório descreve o projeto de engenharia de dados desenvolvido a partir dos microdados de COVID-19 do Estado do Espírito Santo. O projeto abrange desde a ingestão dos dados brutos a partir de um arquivo CSV de grande porte (~1.95 GB) até a modelagem de um Data Warehouse (DW) utilizando PostgreSQL sob a arquitetura Star Schema. Adicionalmente, são analisados os scripts de ETL implementados em SQL e Python, bem como as otimizações físicas e lógicas adotadas para viabilizar consultas analíticas rápidas sobre a base de dados.

Conteúdo

1	Introdução e Objetivos	3
2	Arquitetura e Modelagem do Data Warehouse	3
2.1	Esquema de Staging	3
2.2	Esquema Dimensional (DW)	3
2.3	Estatísticas Físicas do Data Warehouse	4
3	Descrição dos Scripts de Ingestão e ETL	4
3.1	Script SQL Principal (<code>Script.sql</code>)	4
3.2	Script de Carga de Dimensões em Python (<code>etl_dimensoes.py</code>) . . .	5
3.3	Script de Transformação e Carga da Fato (<code>dw_fato.sql</code>)	5
3.4	Validações Analíticas (<code>validacoes.sql</code>)	6
4	Análise de Performance e Otimizações	6
4.1	O Gargalo do CSV e o Impacto do Tamanho do Arquivo	6
4.2	Otimização no PostgreSQL: Staging vs. Data Warehouse	7
5	Conclusão	7

1 Introdução e Objetivos

O volume massivo de dados gerados durante a pandemia de COVID-19 exige abordagens eficientes para armazenamento, processamento e visualização de informações. O conjunto de dados original utilizado neste projeto, obtido da base pública de microdados do Espírito Santo (`MICRODADOS.csv`), possui um tamanho aproximado de **1.95 GB** e contém **5.187.470 registros** de notificações.

Tentar realizar análises diretamente sobre o arquivo CSV bruto ou por meio de tabelas de staging desestruturadas gera sérios problemas de performance. Os principais objetivos deste projeto consistem em:

- Projetar e implementar um **Data Warehouse** robusto em PostgreSQL que armazene os dados de forma estruturada e limpa.
- Desenvolver rotinas de **Extração, Transformação e Carga (ETL)** utilizando scripts SQL nativos e Python com SQLAlchemy para popular dimensões de forma idempotente.
- Implementar **otimizações físicas e lógicas** (como índices e chaves numéricas substitutas) para acelerar consultas analíticas (OLAP) no banco de dados.

2 Arquitetura e Modelagem do Data Warehouse

Para viabilizar consultas rápidas e separar as preocupações analíticas das operacionais, foi projetado um modelo multidimensional em **Star Schema** (Esquema Estrela) no PostgreSQL, estruturado em três schemas principais: **stg** (staging), **dw** (data warehouse) e **mart** (visualizações prontas).

2.1 Esquema de Staging

A tabela `stg.notificacao_raw` funciona como uma área temporária de transição. Ela reflete a estrutura exata do CSV bruto, com todas as colunas importadas como tipo `TEXT`. Isso garante que o processo de ingestão inicial (via comando `COPY`) seja extremamente rápido e não falhe por incompatibilidade de tipos de dados.

2.2 Esquema Dimensional (DW)

A modelagem conta com sete tabelas de dimensões e uma tabela fato centralizada:

1. **dim_tempo**: Dimensão compartilhada (role-playing) que mapeia datas de notificação, cadastro, diagnóstico, coleta, encerramento e óbito. Possui a chave natural calculada como número (YYYYMMDD) e atributos temporais como ano, mês, dia, trimestre, semana epidemiológica e flag de fim de semana.
2. **dim_localidade**: Contém as combinações de município e bairro. Mapeia a hierarquia geográfica regional do Espírito Santo (município, bairro, UF, região e macrorregião).

3. **dim_perfil_paciente**: Reúne dados demográficos do paciente (sexo, faixa etária, raça/cor, escolaridade, gestante, profissional de saúde, morador de rua e possui deficiência).
4. **dim_classificacao**: Mapeia o desfecho clínico do caso (classificação final, evolução do caso, critério de confirmação e status da notificação).
5. **dim_sintomas**: Junk dimension contendo a combinação de sintomas comuns relatados (febre, dificuldade respiratória, tosse, coriza, dor de garganta, diarreia e cefaleia).
6. **dim_comorbidade**: Junk dimension contendo o histórico de doenças prévias do paciente (pulmão, cardio, renal, diabetes, tabagismo e obesidade).
7. **dim_teste**: Detalha os exames realizados (tipo de teste rápido, resultado RT-PCR, resultado teste rápido, resultado sorologia e resultado sorologia IgG).
8. **fato_notificacao_covid**: Tabela central contendo chaves estrangeiras inteiras (INT) apontando para as dimensões correspondentes, além de medidas numéricas otimizadas:
 - **qtd_notificacao**: Medida básica de contagem de registros (igual a 1).
 - **flag_confirmado**, **flag_obito_covid**, **flag_internado**, **flag_cura**: Flags binárias (0 ou 1) pré-calculadas para agilização das somas analíticas.
 - **idade_anos**: Idade do paciente extraída e normalizada para inteiro.
 - **dias_notif_encerramento**: Métrica de tempo calculado entre a data da notificação e o encerramento do caso.
 - **dias_notif_obito**: Métrica de tempo calculado entre a data da notificação e o óbito do paciente.

2.3 Estatísticas Físicas do Data Warehouse

Abaixo são apresentados os números de registros exatos após a carga completa do banco de dados PostgreSQL. O tamanho total ocupado pelo banco de dados em disco é de **3039 MB** (~3.0 GB).

3 Descrição dos Scripts de Ingestão e ETL

O pipeline foi desenvolvido de maneira híbrida, aproveitando o poder de processamento do banco de dados relacional para grandes volumes e a flexibilidade do Python para o controle de cargas e tratamentos específicos.

3.1 Script SQL Principal (Script.sql)

Este script realiza as seguintes etapas fundamentais:

1. Criação dos Schemas (**stg**, **dw**, **mart**) e estruturas de tabelas.

Tabela 1: Quantidade de Linhas por Tabela no Data Warehouse

Schema	Tabela	Quantidade de Linhas
stg	notificacao_raw	5.187.470
dw	fato_notificacao_covid	5.187.470
dw	dim_perfil_paciente	16.282
dw	dim_localidade	8.026
dw	dim_tempo	2.558
dw	dim_sintomas	593
dw	dim_teste	525
dw	dim_comorbidade	300
dw	dim_classificacao	120

2. Importação via COPY a partir do arquivo CSV original.
3. Geração em lote da dimensão tempo por meio de uma `generate_series` cobrindo o período de 2020 a 2026.
4. Carga por inserção distinta (`INSERT INTO ... SELECT DISTINCT ... ON CONFLICT DO NOTHING`) das dimensões descritivas diretamente da staging.
5. Primeira carga da tabela fato estabelecendo os joins básicos.

3.2 Script de Carga de Dimensões em Python (`etl_dimensoes.py`)

Este script foi construído em Python utilizando a biblioteca `SQLAlchemy` e o conector `psycopg2`. Ele encapsula a lógica de carga incremental/idempotente das dimensões por meio de consultas parametrizadas. A função central `carregar_dimensao` lê as combinações distintas da staging e executa instruções SQL tratadas com comandos `ON CONFLICT DO NOTHING` para evitar duplicação de dados, normalizando valores nulos ou vazios para strings padrão como “Desconhecido” ou “Desconhecida”.

3.3 Script de Transformação e Carga da Fato (`dw_fato.sql`)

A tabela fato necessita de validações estritas de dados antes de sua carga para evitar inconsistências nos relatórios. O script `dw_fato.sql` implementa regras cruciais de saneamento de dados:

- **Consistência de Datas:** As datas de notificação, cadastro, diagnóstico, coleta, encerramento e óbito são verificadas. Se estiverem vazias ou fora de um intervalo lógico admissível (entre `'2020-01-01'` e `'2026-12-31'`), elas são mapeadas para o registro neutro `-1` na dimensão tempo.
- **Pre-computação de Flags:** Converte descrições textuais variadas em representações booleanas simples. Por exemplo, a evolução clínica contendo termos como “bito pelo COVID” é padronizada na flag `flag_obito_covid = 1`.

- **Cálculo de Diferença de Datas:** O número de dias decorridos é calculado de maneira segura apenas para datas válidas que estejam dentro do período determinado, evitando erros de overflow de data e facilitando análises de sobrevida dos pacientes.

3.4 Validações Analíticas (`validacoes.sql`)

Reúne cinco consultas analíticas complexas utilizadas para extrair insights da pandemia:

- **Q1:** Quantidade de casos confirmados e total de notificações agrupados por município e mês para os anos de 2021 e 2022.
- **Q2:** Taxa de letalidade percentual calculada para cada faixa etária de pacientes.
- **Q3:** Ranking dos 10 agrupamentos de sintomas corporais mais associados a internações hospitalares.
- **Q4:** Tempo médio em dias entre a notificação e o encerramento do caso agrupado por município.
- **Q5:** Análise de letalidade cruzando diferentes comorbidades graves (problemas cardíacos, diabetes e obesidade).

4 Análise de Performance e Otimizações

Nesta seção, discute-se o impacto real de se trabalhar diretamente com arquivos planos (como o CSV bruto) em comparação com as otimizações implementadas na modelagem multidimensional do PostgreSQL.

4.1 O Gargalo do CSV e o Impacto do Tamanho do Arquivo

O uso de arquivos CSV brutos para conjuntos de dados de grande porte (`MICRODADOS.csv` com 1.95 GB e ~5.18M de linhas) impõe limites severos para o banco de dados:

- **Ingestão de Dados:** No banco de dados PostgreSQL, a carga inicial do arquivo CSV para a tabela de staging usando o utilitário nativo `COPY` demora entre **60 a 90 segundos**.
- **Varreduras de Texto Ineficientes:** Embora o comando `COPY` seja otimizado para carregar os dados brutos, realizar consultas analíticas e processamentos diretamente sobre dados textuais não estruturados em uma tabela com mais de 5 milhões de linhas é inviável, gerando tempos de resposta incompatíveis com ambientes de análise.

4.2 Otimização no PostgreSQL: Staging vs. Data Warehouse

Para comprovar o ganho de desempenho obtido com a modelagem multidimensional (Star Schema), foram realizados testes comparativos executando a mesma regra analítica (comorbidades versus letalidade, consulta Q5) sobre a tabela de staging desordenada e sobre o Data Warehouse estruturado. Os resultados médios obtidos estão descritos na Tabela 2.

Tabela 2: Comparação de Tempo de Resposta de Consultas (5.18M de Registros)

Ambiente	Tipo de Consulta / Operação	Tempo Médio
Staging (Raw)	Consulta Complexa (Q5) com varredura textual	2.154,61 ms
Data Warehouse	Consulta Equivalente (Q5) via Star Schema	1.166,21 ms
Data Warehouse	Consulta de Letalidade por Faixa Etária (Q2)	872,66 ms
Data Warehouse	Consulta de Evolução por Município/Mês (Q1)	1.330,71 ms

A consulta sobre o Data Warehouse obteve um tempo de resposta de apenas **1,16 segundo**, representando uma **redução de aproximadamente 46%** no tempo de processamento comparado à tabela de staging. A performance superior do DW deve-se aos seguintes fatores:

- **Substituição de String por Inteiro:** As junções são feitas comparando números inteiros (chaves substitutas `sk_local`, `sk_perfil`, etc.), que são incomparavelmente mais rápidas de processar que junções baseadas em strings textuais longas.
- **Cálculo Prévio de Métricas:** O uso de flags binárias pré-calculadas na Fato (`flag_confirmado`, `flag_obito_covid`) elimina a necessidade de avaliar expressões condicionais caras como `evolucao ILIKE '%bito%'` ou `classificacao = 'Confirmados'` em tempo de execução para os 5.18 milhões de registros.
- **Índices B-tree direcionados:** A criação de índices dedicados nas colunas chaves da Fato (`sk_data_notificacao`, `sk_local`, `sk_class`, `sk_perfil`) permite que o planejador de consultas filtre os registros rapidamente, sem a necessidade de realizar varreduras sequenciais completas (*Sequential Scans*) na tabela fato inteira.

5 Conclusão

A implementação de uma arquitetura de Data Warehouse estruturada em Star Schema no PostgreSQL provou-se altamente eficaz, acelerando de forma significativa a execução de consultas complexas em uma base com mais de 5 milhões de linhas. O uso correto de chaves numéricas substitutas (chaves substitutas inteiras), índices focados em OLAP e métricas pré-calculadas na tabela fato mitigou os sérios problemas de performance e tempo de resposta associados à leitura direta e à agregação de dados brutos textuais em formato CSV.